

Laporan Tugas Besar

IF2224 TEORI BAHASA FORMAL DAN OTOMATA

MILESTONE 2: SYNTAX ANALYSIS

SEMESTER II / 2025–2026



Disusun oleh:

GTFOTBFO (HBB)

Nathanael Imandatua Manurung - 13524021

Nicholas Wise Saragih Sumbayak - 13524037

Kurt Mikhael Purba - 13524065

Rava Khoman Tuah Saragih - 13524107

LABORATORIUM ILMU DAN REKAYASA KOMPUTASI

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

Daftar Isi

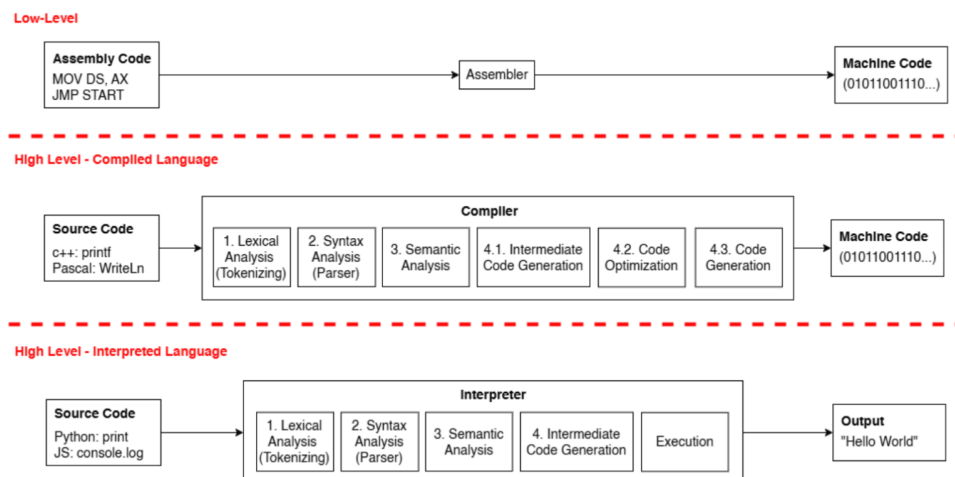
1	Pendahuluan	3
1.1	Latar Belakang	3
1.2	Rumusan Masalah	4
1.3	Gambaran Umum Sistem	4
1.4	Spesifikasi Umum	4
2	Dasar Teori	5
2.1	Pendefinisian Bahasa Pemrograman	5
2.1.1	Hierarki Bahasa Pemrograman	5
2.1.2	<i>Compiler</i> dan <i>Interpreter</i>	5
2.2	Teori Kompilator (<i>Compiler Theory</i>)	6
2.2.1	Analisis Sintaksis (<i>Syntax Analysis</i>)	6
2.3	Teori Bahasa Formal dan Otomata	7
2.3.1	Context-Free Grammar (CFG)	7
2.3.2	Derivasi dan Parse Tree	8
2.3.3	Recursive Descent Parser	8
3	Perancangan Syntax Analyzer	8
3.1	Grammar Bahasa Arion	9
3.1.1	Struktur Program	9
3.1.2	Deklarasi	9
3.1.3	Subprogram	9
3.1.4	Statement	10
3.1.5	Ekspresi	10
3.1.6	Variable	10
3.2	Perancangan Parse Tree	10
3.3	Perancangan Algoritma Recursive Descent	11
3.4	Strategi Penanganan Kesalahan	11
3.5	Alur Program	12
4	Implementasi Parser	12
4.1	Struktur Kelas Parser	12
4.2	Struktur Data Parse Tree	13
4.3	Implementasi Aturan Grammar	14
4.3.1	Program dan Struktur Dasar	14
4.3.2	Implementasi Deklarasi	15
4.3.3	Implementasi Statement	16
4.3.4	Implementasi Ekspresi	17
4.4	Mekanisme Disambiguasi	17
4.5	Penanganan Kesalahan Sintaks	18
4.6	Pencetakan dan Penyimpanan Parse Tree	19
4.7	Struktur Direktori Proyek	20
5	Pengujian	20
5.1	Kasus Uji 1: Kata Kunci dalam Berbagai Huruf Besar/Kecil	21
5.2	Kasus Uji 2: Ambiguitas Bilangan Riil vs Titik	21
5.3	Kasus Uji 3: Charcon vs String	22

5.4	Kasus Uji 4: Komentar Bersarang dan Berkas Hanya Komentar	23
5.5	Kasus Uji 5: Konstruksi yang Tidak Ditutup	24
5.6	Kasus Uji 6: Operator == vs Karakter = Tunggal	25
5.7	Kasus Uji 7: Operator := vs : Diikuti Karakter Lain	26
5.8	Ringkasan Hasil Pengujian	27
6	Akhiran	28
6.1	Kesimpulan	28
6.2	Tantangan yang Dihadapi	28
	References	29
	Appendix	29
	Pembagian Tugas	29

1 Pendahuluan

Pada tahap sebelumnya, telah berhasil dibangun sebuah *lexical analyzer* (lexer) yang mampu membaca kode sumber bahasa Arion dan mengubahnya menjadi daftar token. Setiap token merepresentasikan unit leksikal terkecil dari program, seperti kata kunci, identifer, konstanta, dan operator. Meskipun lexer telah berfungsi dengan baik dalam mengenali dan mengklasifikasikan token-token tersebut, lexer belum mampu memastikan apakah urutan token tersebut membentuk struktur program yang valid sesuai dengan aturan tata bahasa (*grammar*) bahasa Arion.

Proses transformasi kode sumber menjadi program yang dapat dijalankan melibatkan beberapa tahap berurutan. Setelah analisis leksikal selesai, tahap berikutnya adalah analisis sintaktik (*syntax analysis*) atau yang sering disebut *parsing*. Pada tahap ini, parser menerima daftar token dari lexer dan memeriksa apakah urutan token tersebut sesuai dengan aturan grammar yang didefinisikan untuk bahasa Arion. Parser kemudian membangun sebuah struktur hierarkis yang disebut Parse Tree, yang merepresentasikan hubungan struktural antar-komponen program secara menyeluruh.



Gambar 1: Tahap-tahap kompilasi dalam compiler

Sumber: Asisten IF2224

Parse Tree adalah representasi lengkap dari struktur sintaks program sesuai dengan grammar bahasa yang digunakan. Setiap node dalam parse tree merepresentasikan simbol non-terminal atau terminal dari aturan produksi grammar. Berbeda dengan Abstract Syntax Tree (AST) yang menyederhanakan struktur dengan menghapus node-node informatif, Parse Tree memuat semua detail simbol seolah-olah seseorang menggambarkan semua langkah produksi secara lengkap. Parse Tree berguna untuk menunjukkan bagaimana token-token hasil lexical analysis membentuk struktur program yang sesuai dengan aturan grammar.

1.1 Latar Belakang

Proses kompilasi suatu bahasa pemrograman terdiri atas beberapa tahap yang saling berkaitan, dimulai dari analisis leksikal (*lexical analysis*), analisis sintaktik, analisis semantik, hingga pembangkitan kode (*code generation*). Setelah lexer berhasil mengonversi kode sumber menjadi deretan token, tahap analisis sintaktik menjadi penting untuk

memverifikasi apakah urutan token tersebut memenuhi aturan tata bahasa yang telah didefinisikan.

Pada tugas besar mata kuliah IF2224 Teori Bahasa Formal dan Otomata, mahasiswa diminta untuk membangun sebuah *compiler* sederhana untuk bahasa pemrograman Arion, yaitu bahasa bertipe Pascal. Milestone kedua ini berfokus pada pembangunan *syntax analyzer* (parser) yang mampu memeriksa urutan token dari lexer dan membangun Parse Tree sesuai dengan aturan grammar bahasa Arion. Parser yang dibangun menggunakan algoritma *Recursive Descent* yang mengimplementasikan setiap aturan produksi dalam grammar sebagai fungsi-fungsi rekursif.

1.2 Rumusan Masalah

Bagaimana merancang dan mengimplementasikan sebuah *syntax analyzer* berbasis *Recursive Descent Parser* yang mampu:

1. Membaca daftar token dari keluaran *lexical analyzer*.
2. Memeriksa apakah urutan token sesuai dengan aturan tata bahasa (*grammar*) bahasa Arion.
3. Mendeteksi dan melaporkan kesalahan sintaks (*syntax error*) dengan informasi lokasi yang akurat.
4. Membangun Parse Tree yang merepresentasikan struktur hierarkis program sumber.

1.3 Gambaran Umum Sistem

Sistem yang dibangun merupakan sebuah *syntax analyzer* untuk bahasa Arion yang diimplementasikan dalam bahasa C++20. Parser ini menerima daftar token dari lexer (berupa berkas teks yang berisi token-token), memproses setiap token menggunakan algoritma Recursive Descent, dan menghasilkan Parse Tree sebagai keluaran. Sistem terdiri atas beberapa komponen utama: Parser (mesin Recursive Descent), Node (struktur data node pada Parse Tree), TreeBuilder (konstruktor Parse Tree), dan ErrorHandler (pelaporan kesalahan sintaks).

1.4 Spesifikasi Umum

1. Menggunakan algoritma *Recursive Descent* dengan grammar yang dispesifikasikan dalam program.
2. Mendukung seluruh node Parse Tree sesuai spesifikasi: program, program-header, declaration-part, compound-statement, statement, expression, dan lainnya.
3. Melaporkan kesalahan sintaks dengan pesan yang informatif dan akurat.
4. Menghasilkan Parse Tree yang terformat dengan jelas, ditampilkan ke terminal dan disimpan ke dalam berkas teks.
5. Menerima masukan berupa berkas daftar token hasil dari lexer milestone pertama.

2 Dasar Teori

2.1 Pendefinisian Bahasa Pemrograman

Bahasa Pemrograman adalah cara manusia untuk dapat memberikan perintah (*instruction*) kepada sebuah komputer. Setiap bahasa pemrograman memiliki kata kunci dan aturannya masing-masing serta makna dari kata-kata itu yang disebut sintaks(*syntax*) dan semantik (*semantics*). Input yang terdapat kesalahan pada tingkat sintaks berarti input tersebut tidak valid untuk bahasa pemrograman tersebut, sedangkan kesalahan pada tingkat semantik umumnya berbentuk kesalahan logika.

2.1.1 Hierarki Bahasa Pemrograman

Berdasarkan tingkat abstraksinya, bahasa pemrograman diklasifikasikan menjadi dua kategori utama:

- ***Low-Level Language (Bahasa Tingkat Rendah)***: Bahasa yang instruksinya mendekati arsitektur perangkat keras dan bahasa mesin (biner 0 dan 1). Contohnya adalah *Assembly*, yang memerlukan *Assembler* untuk menerjemahkan kode menjadi instruksi yang dapat dipahami CPU.
- ***High-Level Language (Bahasa Tingkat Tinggi)***: Bahasa yang dirancang agar lebih mudah dibaca dan dipahami oleh manusia karena menggunakan kosakata yang menyerupai bahasa natural. Semakin tinggi tingkat bahasa tersebut, semakin banyak tahapan pemrosesan yang diperlukan untuk mengubahnya menjadi kode mesin. Contoh bahasa tingkat tinggi antara lain Pascal, C++, Python, dan Java.

2.1.2 *Compiler* dan *Interpreter*

Untuk menjalankan kode sumber yang ditulis dalam bahasa tingkat tinggi, diperlukan sebuah perangkat lunak penerjemah yang mengubah kode tersebut menjadi format yang dapat dieksekusi oleh mesin. Secara umum, terdapat dua mekanisme utama untuk mengubah bahasa tingkat tinggi ke bahasa mesin:

1. ***Compiler (Kompilator)***: *Compiler* adalah program yang menerjemahkan seluruh *source code* secara utuh menjadi kode mesin (biasanya dalam bentuk *Object File*) sebelum program dijalankan. Proses kompilasi pada *compiler* umumnya terdiri dari empat tahapan utama: *Lexical Analysis*, *Syntax Analysis*, *Semantic Analysis*, dan *Intermediate Code Generation*. Karakteristik utama *compiler* adalah jika terdeteksi adanya kesalahan (*error*) di dalam *source code*, maka seluruh proses kompilasi akan gagal dan program tidak dapat dijalankan.
2. ***Interpreter***: Berbeda dengan *compiler*, *interpreter* memproses kode sumber baris demi baris secara langsung. *Interpreter* membaca, menerjemahkan, dan langsung mengeksekusi kode mesin tersebut baris demi baris. Hal ini memungkinkan program tetap berjalan normal hingga baris sebelum ditemukannya kesalahan; misalnya, jika terdapat *error* pada baris ke-10, baris 1 sampai 9 akan tetap berhasil dieksekusi. Meskipun mekanisme eksekusinya berbeda, *interpreter* sebenarnya memiliki tahapan pemrosesan internal yang serupa dengan *compiler*.

2.2 Teori Kompilator (*Compiler Theory*)

Proses kompilasi adalah urutan dari berbagai fase (*phases*) yang bekerja secara berurutan. Setiap fase mengambil input dari tahap sebelumnya, memprosesnya, dan memberikan output yang akan digunakan sebagai input untuk fase berikutnya. Secara umum, arsitektur sebuah kompilator terdiri dari fase-fase berikut:

1. ***Lexical Analysis***: Fase pertama yang bekerja sebagai pemindai teks (*text scanner*) pada kode sumber. Penjelasan lebih detail mengenai fase ini dibahas pada sub-bab tersendiri di bawah.
2. ***Syntax Analysis (Parsing)***: Mengambil *token* yang dihasilkan oleh *lexical analysis* dan membangun pohon sintaksis (*parse tree*). Fase ini memeriksa kebenaran susunan *token* berdasarkan aturan tata bahasa (*grammar*) kode sumber.
3. ***Semantic Analysis***: Memeriksa apakah pohon sintaksis yang dibangun mengikuti aturan semantik bahasa, seperti kecocokan operasi tipe data dan memastikan *identifier* telah dideklarasikan sebelum digunakan. Fase ini menghasilkan keluaran berupa pohon sintaksis yang dianotasi.
4. ***Intermediate Code Generation***: Menghasilkan representasi kode perantara untuk mesin abstrak, yang posisinya menjembatani antara bahasa tingkat tinggi dan bahasa mesin.
5. ***Code Optimization***: Mengoptimalkan kode perantara dengan menghapus baris kode yang tidak perlu dan mengatur ulang urutan instruksi. Tujuannya adalah untuk mempercepat eksekusi program tanpa membuang sumber daya (*CPU*, memori).
6. ***Target Code Generation***: Merupakan fase sintesis akhir (*back-end*) yang menggunakan representasi kode perantara dan tabel simbol untuk menghasilkan program target atau bahasa mesin.

2.2.1 Analisis Sintaksis (*Syntax Analysis*)

Sebagai fokus utama pada *milestone* ini, Analisis Sintaksis atau *parser* adalah tahapan atau fase kedua dari sebuah kompilator. Fase ini bertugas untuk memeriksa urutan *token* yang dihasilkan oleh *lexical analyzer* agar sesuai dengan aturan tata bahasa (*grammar*) dari bahasa pemrograman yang dikompilasi. Dalam mendefinisikan dan melakukan evaluasi aturan sintaks, *parser* secara umum mengadopsi format tata bahasa bebas konteks atau *Context-Free Grammar* (CFG).

Berdasarkan prinsip desain kompilator, mekanisme kerja dan tahapan *parser* bertujuan untuk:

- **Memvalidasi Struktur Sintaks**: Memastikan urutan *token* membentuk struktur sintaks yang valid.
- **Penanganan Kesalahan Sintaks (*Syntax Error*)**: Mendeteksi dan melaporkan kesalahan sintaks (*syntax error*) apabila ditemukan ketidaksesuaian.
- **Membangun Representasi Program**: Mempersiapkan representasi program untuk tahap kompilasi berikutnya. *Parser* akan membangun struktur sintaks hierarkis seperti *Parse Tree* atau *Abstract Syntax Tree* (AST).

Parse Tree adalah representasi lengkap dari struktur sintaks program sesuai dengan *grammar* bahasa yang digunakan. Karena merepresentasikan hierarki secara utuh, setiap *node* di dalam pohon ini memuat semua simbol terminal dan non-terminal. *Parse tree* berguna untuk menunjukkan bagaimana *token-token* hasil *lexical analysis* membentuk struktur program yang sesuai dengan aturan *grammar*.

Secara garis besar, pendekatan *syntax analyzer* dapat diklasifikasikan ke dalam dua jenis *parsing*:

- **Top-Down Parsing:** Pendekatan konstruksi *parse tree* yang dimulai dari simpul akar (*root*) dan diturunkan ke arah simpul daun (*leaves*). Implementasi yang umum digunakan pada pendekatan ini adalah algoritma *Recursive Descent*.
- **Bottom-Up Parsing:** Pendekatan konstruksi *parse tree* yang dimulai dari simpul daun (*leaves*) dan dikompresi perlahan menuju simpul akar (*root*). Contoh algoritma dari kategori ini adalah *LR Parser*.

2.3 Teori Bahasa Formal dan Otomata

Berbeda dengan tahap analisis leksikal yang menggunakan *Regular Expression* dan *Finite Automata* untuk mengenali pola token, tahap analisis sintaktik menggunakan konsep (*Context-Free Grammar*) sebagai dasar untuk memeriksa dan membangun struktur program.

2.3.1 Context-Free Grammar (CFG)

Context-Free Grammar (CFG) adalah formalisme matematis yang digunakan untuk mendefinisikan sintaks dari bahasa pemrograman. CFG terdiri dari himpunan simbol terminal, himpunan simbol non-terminal, himpunan aturan produksi, dan simbol awal. Aturan produksi menentukan bagaimana simbol non-terminal dapat diturunkan menjadi rangkaian simbol terminal dan non-terminal lainnya.

Secara formal, sebuah CFG didefinisikan sebagai *4-tuple* (V, T, P, S) , yang terdiri dari:

- V : Himpunan hingga simbol non-terminal (variabel).
- T : Himpunan hingga simbol terminal (token dari lexer).
- P : Himpunan aturan produksi yang mendefinisikan bagaimana simbol diturunkan.
- S : Simbol awal (*start symbol*), di mana $S \in V$.

Dalam konteks compiler, simbol terminal berasal dari hasil analisis leksikal (token), sedangkan simbol non-terminal merepresentasikan struktur sintaks program seperti `<program>`, `<statement>`, `<expression>`, dan lainnya.

Sebagai contoh, grammar sederhana untuk ekspresi aritmatika dapat didefinisikan sebagai:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T \times F \mid F \\ F &\rightarrow (E) \mid \text{intcon} \end{aligned}$$

Aturan produksi ini menunjukkan bahwa sebuah ekspresi E dapat berupa $E + T$ atau simplemente T , dan seterusnya.

2.3.2 Derivasi dan Parse Tree

Proses penerapan aturan produksi untuk mengubah simbol awal menjadi deretan simbol terminal disebut *derivasi*. Terdapat dua strategi derivasi utama:

- ***Leftmost Derivation***: Simbol non-terminal paling kiri dikembangkan terlebih dahulu pada setiap langkah.
- ***Rightmost Derivation***: Simbol non-terminal paling kanan dikembangkan terlebih dahulu pada setiap langkah.

Parse Tree adalah representasi visual dari proses derivasi. Setiap node dalam parse tree merepresentasikan simbol (terminal atau non-terminal), dan setiap edge merepresentasikan penerapan aturan produksi. Parse Tree memuat seluruh simbol yang muncul selama derivasi, berbeda dengan Abstract Syntax Tree (AST) yang menghilangkan node-node informatif.

2.3.3 Recursive Descent Parser

Recursive Descent adalah algoritma *top-down parsing* yang mengimplementasikan setiap aturan produksi dalam grammar sebagai fungsi rekursif. Setiap fungsi bertanggung jawab untuk mengenali simbol non-terminal tertentu dan semua turunannya.

Prinsip kerja Recursive Descent adalah:

1. Setiap simbol non-terminal diimplementasikan sebagai satu fungsi.
2. Fungsi membaca token dari input dan memeriksa apakah token tersebut sesuai dengan ekspektasi berdasarkan aturan produksi.
3. Jika ditemukan simbol non-terminal di ruas kanan produksi, fungsi memanggil fungsi lain yang berhubungan dengannya.
4. Jika token yang dibaca tidak sesuai, parser melaporkan kesalahan sintaks.

Keunggulan Recursive Descent adalah kesederhanaan implementasinya dan kemudahan dalam memahami alur parsing. Namun, kelemahannya adalah ketidakmampuan untuk menangani *left recursion* (produk seperti $A \rightarrow A\alpha$) yang dapat menyebabkan infinite recursion.

3 Perancangan Syntax Analyzer

Perancangan *syntax analyzer* untuk bahasa Arion didasarkan pada aturan tata bahasa bebas konteks (*Context-Free Grammar*) yang mendefinisikan struktur sintaks bahasa secara hierarkis. Parser yang dibangun menggunakan algoritma *Recursive Descent* dengan pendekatan *top-down parsing*. Pada bagian ini, dijelaskan rancangan grammar bahasa Arion, perancangan Parse Tree, algoritma Recursive Descent, strategi penanganan kesalahan, dan alur kerja program.

3.1 Grammar Bahasa Arion

Grammar bahasa Arion didefinisikan dalam bentuk aturan produksi (*production rules*) yang terdiri atas simbol non-terminal (struktur sintaks) dan simbol terminal (token hasil lexer). Simbol awal (*start symbol*) dari grammar ini adalah **<program>**. Berikut adalah seluruh aturan produksi yang digunakan dalam perancangan parser.

3.1.1 Struktur Program

$$\begin{aligned}\langle \text{program} \rangle &\rightarrow \langle \text{program-header} \rangle \langle \text{declaration-part} \rangle \langle \text{compound-statement} \rangle \text{period} \\ \langle \text{program-header} \rangle &\rightarrow \text{programsy ident semicolon} \\ \langle \text{block} \rangle &\rightarrow \langle \text{declaration-part} \rangle \langle \text{compound-statement} \rangle\end{aligned}$$

3.1.2 Deklarasi

$$\begin{aligned}\langle \text{declaration-part} \rangle &\rightarrow (\langle \text{const-declaration} \rangle)^* (\langle \text{type-declaration} \rangle)^* \\ &\quad (\langle \text{var-declaration} \rangle)^* (\langle \text{subprogram-declaration} \rangle)^* \\ \langle \text{const-declaration} \rangle &\rightarrow \text{constsy (ident eql \langle constant \rangle semicolon)}^+ \\ \langle \text{constant} \rangle &\rightarrow \text{charcon} \mid \text{string} \mid [\text{plus} \mid \text{minus}] (\text{ident} \mid \text{intcon} \mid \text{realcon}) \\ \langle \text{type-declaration} \rangle &\rightarrow \text{typesy (ident eql \langle type \rangle semicolon)}^+ \\ \langle \text{type} \rangle &\rightarrow \text{ident} \mid \langle \text{array-type} \rangle \mid \langle \text{range} \rangle \mid \langle \text{enumerated} \rangle \mid \langle \text{record-type} \rangle \\ \langle \text{array-type} \rangle &\rightarrow \text{arraysy lbrack} (\langle \text{range} \rangle \mid \text{ident}) \text{rbrack ofsy} \langle \text{type} \rangle \\ \langle \text{range} \rangle &\rightarrow \langle \text{expression} \rangle \text{period period} \langle \text{expression} \rangle \\ \langle \text{enumerated} \rangle &\rightarrow \text{lparent ident (comma ident)}^* \text{rparent} \\ \langle \text{record-type} \rangle &\rightarrow \text{recordsy} \langle \text{field-list} \rangle \text{endsy} \\ \langle \text{field-list} \rangle &\rightarrow \langle \text{field-part} \rangle (\text{semicolon} \langle \text{field-part} \rangle)^* \\ \langle \text{field-part} \rangle &\rightarrow \langle \text{identifier-list} \rangle \text{colon} \langle \text{type} \rangle \\ \langle \text{var-declaration} \rangle &\rightarrow \text{varsy} (\langle \text{identifier-list} \rangle \text{colon} \langle \text{type} \rangle \text{semicolon})^+ \\ \langle \text{identifier-list} \rangle &\rightarrow \text{ident (comma ident)}^*\end{aligned}$$

3.1.3 Subprogram

$$\begin{aligned}\langle \text{subprogram-declaration} \rangle &\rightarrow \langle \text{procedure-declaration} \rangle \mid \langle \text{function-declaration} \rangle \\ \langle \text{procedure-declaration} \rangle &\rightarrow \text{proceduresy ident} [\langle \text{formal-parameter-list} \rangle] \text{semicolon} \\ &\quad \langle \text{block} \rangle \text{semicolon} \\ \langle \text{function-declaration} \rangle &\rightarrow \text{functionsy ident} [\langle \text{formal-parameter-list} \rangle] \text{colon ident} \\ &\quad \text{semicolon} \langle \text{block} \rangle \text{semicolon} \\ \langle \text{formal-parameter-list} \rangle &\rightarrow \text{lparent} \langle \text{parameter-group} \rangle (\text{semicolon} \langle \text{parameter-group} \rangle)^* \text{rparent} \\ \langle \text{parameter-group} \rangle &\rightarrow \langle \text{identifier-list} \rangle \text{colon} (\text{ident} \mid \langle \text{array-type} \rangle)\end{aligned}$$

3.1.4 Statement

$$\begin{aligned}
\langle \text{compound-statement} \rangle &\rightarrow \text{beginsy } \langle \text{statement-list} \rangle \text{ endsy} \\
\langle \text{statement-list} \rangle &\rightarrow \langle \text{statement} \rangle (\text{semicolon } \langle \text{statement} \rangle)^* \\
\langle \text{statement} \rangle &\rightarrow \langle \text{assignment-statement} \rangle \mid \langle \text{if-statement} \rangle \mid \langle \text{case-statement} \rangle \\
&\mid \langle \text{while-statement} \rangle \mid \langle \text{repeat-statement} \rangle \mid \langle \text{for-statement} \rangle \\
&\mid \langle \text{procedure/function-call} \rangle \mid \epsilon \\
\langle \text{assignment-statement} \rangle &\rightarrow \langle \text{variable} \rangle \text{ becomes } \langle \text{expression} \rangle \\
\langle \text{if-statement} \rangle &\rightarrow \text{ifsy } \langle \text{expression} \rangle \text{ then } \langle \text{statement} \rangle \\
&\quad [\text{elsesy } \langle \text{statement} \rangle] \\
\langle \text{case-statement} \rangle &\rightarrow \text{casesy } \langle \text{expression} \rangle \text{ ofsy } \langle \text{case-block} \rangle \text{ endsy} \\
\langle \text{case-block} \rangle &\rightarrow \langle \text{constant} \rangle (\text{comma } \langle \text{constant} \rangle)^* \text{ colon } \langle \text{statement} \rangle \\
&\quad (\text{semicolon } \langle \text{case-block} \rangle)^* \\
\langle \text{while-statement} \rangle &\rightarrow \text{whilesy } \langle \text{expression} \rangle \text{ dosy } \langle \text{statement} \rangle \\
\langle \text{repeat-statement} \rangle &\rightarrow \text{repeatsy } \langle \text{statement-list} \rangle \text{ untilsy } \langle \text{expression} \rangle \\
\langle \text{for-statement} \rangle &\rightarrow \text{forsy ident becomes } \langle \text{expression} \rangle \\
&\quad (\text{tosy} \mid \text{downtosy}) \langle \text{expression} \rangle \text{ dosy } \langle \text{statement} \rangle \\
\langle \text{procedure/function-call} \rangle &\rightarrow \text{ident} [\text{lparent } \langle \text{parameter-list} \rangle \text{ rparent}] \\
\langle \text{parameter-list} \rangle &\rightarrow \langle \text{expression} \rangle (\text{comma } \langle \text{expression} \rangle)^*
\end{aligned}$$

3.1.5 Ekspresi

$$\begin{aligned}
\langle \text{expression} \rangle &\rightarrow \langle \text{simple-expression} \rangle \\
&\quad [\langle \text{relational-operator} \rangle \langle \text{simple-expression} \rangle] \\
\langle \text{simple-expression} \rangle &\rightarrow [\text{plus} \mid \text{minus}] \langle \text{term} \rangle \\
&\quad (\text{plus} \mid \text{minus} \mid \text{orsy}) \langle \text{term} \rangle^* \\
\langle \text{term} \rangle &\rightarrow \langle \text{factor} \rangle ((\text{times} \mid \text{rdiv} \mid \text{idiv} \mid \text{imod} \mid \text{andsy}) \langle \text{factor} \rangle)^* \\
\langle \text{factor} \rangle &\rightarrow \text{intcon} \mid \text{realcon} \mid \text{charcon} \mid \text{string} \mid \langle \text{variable} \rangle \\
&\quad \mid \langle \text{procedure/function-call} \rangle \mid (\langle \text{expression} \rangle) \mid \text{notsy } \langle \text{factor} \rangle
\end{aligned}$$

3.1.6 Variable

$$\begin{aligned}
\langle \text{variable} \rangle &\rightarrow \text{ident} (\langle \text{component-variable} \rangle)^* \\
\langle \text{component-variable} \rangle &\rightarrow \text{lbrack } \langle \text{index-list} \rangle \text{ rbrack} \mid \text{period ident} \\
\langle \text{index-list} \rangle &\rightarrow (\text{intcon} \mid \text{charcon} \mid \text{ident}) (\text{comma } (\text{intcon} \mid \text{charcon} \mid \text{ident}))^*
\end{aligned}$$

3.2 Perancangan Parse Tree

Parse Tree yang dihasilkan oleh parser merupakan representasi hierarkis dari struktur sintaks program. Setiap simpul (*node*) dalam Parse Tree memuat label yang menunjukkan

simbol terminal atau non-terminal yang direpresentasikan. Simpul non-terminal diberi label dengan nama aturan produksi dalam tanda kurung sudut (misalnya `<program>`, `<expression>`), sedangkan simpul terminal diberi label sesuai representasi token dari lexer (misalnya `programs`, `ident(Hello)`, `semicolon`).

Struktur data yang digunakan untuk merepresentasikan Parse Tree terdiri dari label string yang mengidentifikasi node (terminal atau non-terminal dan daftar anak (sub-pohon) yang terurut sesuai urutan dalam aturan produksi. Setiap node non-terminal dibuat dengan menciptakan node baru, kemudian menambahkan node-node anak sesuai dengan aturan produksi yang sedang diproses. Node terminal dibuat dengan membungkus token yang telah dikonsumsi dari aliran masukan.

3.3 Perancangan Algoritma Recursive Descent

Algoritma Recursive Descent dirancang dengan mengimplementasikan setiap aturan produksi dalam grammar sebagai fungsi rekursif. Setiap fungsi:

1. Membuat node Parse Tree baru dengan label non-terminal yang sesuai.
2. Memeriksa token saat ini (*lookahead*) untuk menentukan cabang produksi yang akan diambil.
3. Mengonsumsi token terminal yang sesuai atau memanggil fungsi non-terminal lain secara rekursif.
4. Mengembalikan node yang telah terisi lengkap dengan anak-anaknya.

Dalam perancangan ini, terdapat beberapa strategi disambiguasi untuk menangani kasus di mana satu token dapat mengawali beberapa aturan produksi berbeda:

- **Statement vs Assignment vs Function Call:** Ketika parser menemukan token `ident`, parser menyimpan posisi saat ini, mencoba mem-parsing sebagai `<variable>`, lalu memeriksa apakah token berikutnya adalah `becomes (:=)`. Jika ya, diinterpretasikan sebagai `<assignment-statement>`; jika tidak, diinterpretasikan sebagai `<procedure/function-call>`.
- **Factor: variable vs function call:** Dalam `<factor>`, saat menemukan `ident`, parser melihat satu token ke depan (*lookahead*) untuk memeriksa apakah terdapat `lparent`. Jika ya, diinterpretasikan sebagai pemanggilan fungsi; jika tidak, sebagai `variable`.
- **Range vs ident dalam type:** Dalam `<type>`, token `ident` bisa merupakan tipe sederhana atau awal dari `<range>`. Parser memeriksa keberadaan dua titik berurutan (`period period`) setelah ekspresi untuk menentukan interpretasi.

3.4 Strategi Penanganan Kesalahan

Parser dirancang untuk mendeteksi dan melaporkan kesalahan sintaks dengan informasi yang informatif. Strategi yang digunakan meliputi:

- **Error detection:** Setiap fungsi parsing memeriksa token yang diharapkan. Jika token tidak sesuai dengan yang diharapkan, parser segera melaporkan kesalahan.

- **Pesan error informatif:** Pesan kesalahan mencakup nomor baris, deskripsi kesalahan, dan token yang ditemukan (misalnya "Syntax error on line 3: Expected ';' after program name (got period)").
- **Error recovery:** Parser menggunakan pendekatan *panic mode*, yaitu melompati token hingga menemukan token sinkronisasi (seperti semicolon, endsy, atau beginsy) untuk melanjutkan parsing.

3.5 Alur Program

Alur kerja program secara keseluruhan adalah sebagai berikut:

1. Program membaca berkas kode sumber dari direktori `test/input/`.
2. **Scanner** melakukan analisis leksikal dan menghasilkan daftar token.
3. Token-token tersebut ditampilkan ke terminal untuk verifikasi.
4. **Parser** menerima daftar token dan melakukan analisis sintaks dengan algoritma Recursive Descent untuk membangun Parse Tree.
5. Parse Tree yang dihasilkan ditampilkan ke terminal dengan format hierarkis dan disimpan ke dalam berkas teks di direktori `test/output/`.

4 Implementasi Parser

Parser pada compiler Arion diimplementasikan menggunakan algoritma Recursive Descent dalam bahasa C++20. Setiap aturan produksi dalam grammar bahasa Arion direpresentasikan sebagai satu fungsi yang mengembalikan node Parse Tree. Implementasi ini terletak pada direktori `src/syntax/` dengan struktur kelas yang terdefinisi dengan baik.

4.1 Struktur Kelas Parser

Kelas `Parser` didefinisikan dalam berkas `Parser.hpp` dan diimplementasikan dalam `Parser.cpp`. Kelas ini menyimpan daftar token sebagai masukan dan sebuah pointer posisi (`pos`) untuk menelusuri token secara sekuensial.

Listing 1: Deklarasi kelas Parser

```
1 class Parser {
2     public:
3         explicit Parser(const std::vector<Token> &tokens);
4         std::shared_ptr<ParseTreeNode> parse();
5
6     private:
7         std::vector<Token> tokens;
8         size_t pos = 0;
9         // ...
10 };
```

Kelas `Parser` menyediakan beberapa metode bantu (*helper methods*) untuk navigasi token yang menjadi fondasi bagi seluruh fungsi parsing:

- `peek()`: Mengembalikan token saat ini tanpa mengonsumsinya.
- `advance()`: Mengonsumsi token saat ini dan memajukan posisi.
- `previous()`: Mengembalikan token yang baru saja dikonsumsi.
- `check(type)`: Memeriksa apakah token saat ini bertipe tertentu.
- `match(type)`: Jika token saat ini sesuai, konsumsi dan kembalikan `true`.
- `expect(type, errMsg)`: Mengonsumsi token dan melaporkan error jika tidak sesuai.
- `consumeTerminal(type, errMsg)`: Seperti `expect`, tetapi membungkus hasilnya dalam node Parse Tree.
- `isAtEnd()`: Memeriksa apakah seluruh token telah diproses.

Listing 2: Implementasi metode bantu parser

```

1  const Token &Parser::peek() const { return tokens[pos]; }
2
3  const Token &Parser::advance() {
4      if (!isAtEnd()) pos++;
5      return previous();
6  }
7
8  bool Parser::match(TokenType type) {
9      if (check(type)) { advance(); return true; }
10     return false;
11 }
12
13 const Token &Parser::expect(TokenType type, const string &
    errMsg) {
14     if (check(type)) return advance();
15     error(errMsg);
16 }
17
18 shared_ptr<ParseTreeNode>
19 Parser::consumeTerminal(TokenType type, const string &errMsg)
    {
20     const Token &tok = expect(type, errMsg);
21     return makeNode(tok.toString());
22 }

```

4.2 Struktur Data Parse Tree

Parse Tree direpresentasikan menggunakan struktur `ParseTreeNode` yang didefinisikan dalam berkas `ParseTree.hpp`. Setiap node hanya menyimpan label string dan daftar anak (sub-pohon) yang terurut. Tidak ada perbedaan tipe node – semua node adalah `ParseTreeNode` dengan label yang menunjukkan simbol terminal atau non-terminal. Node non-terminal diberi label seperti `<program>`, `<expression>`, sedangkan node terminal diberi label sesuai representasi token seperti `programsy`, `ident(Hello)`.

Listing 3: Struktur data ParseTreeNode

```

1 struct ParseTreeNode {
2     std::string label;
3     std::vector<std::shared_ptr<ParseTreeNode>> children;
4
5     explicit ParseTreeNode(const std::string &label) : label(
6         label) {}
7
8     void addChild(std::shared_ptr<ParseTreeNode> child) {
9         children.push_back(std::move(child));
10    }
11
12    void print(std::ostream &os, const std::string &prefix =
13        "",
14              bool isLast = true, bool isRoot = true) const;
15
16    void printToConsole() const;
17    void saveToFile(const std::string &path) const;
18 };
19
20 inline std::shared_ptr<ParseTreeNode> makeNode(const std::
21     string &label) {
22     return std::make_shared<ParseTreeNode>(label);
23 }

```

Fungsi `makeNode()` digunakan sebagai *factory function* untuk membuat node baru. Setiap node dapat memiliki banyak anak yang ditambahkan melalui metode `addChild()`. Node terminal dibuat dengan memanggil `consumeTerminal()` yang membungkus token hasil parsing menjadi node.

4.3 Implementasi Aturan Grammar

Setiap aturan produksi dalam grammar bahasa Arion diimplementasikan sebagai satu fungsi yang mengembalikan `shared_ptr<ParseTreeNode>`. Pola implementasi setiap fungsi adalah sama: membuat node baru dengan label non-terminal, kemudian memanggil fungsi `consumeTerminal()` untuk token terminal atau fungsi parsing lain untuk non-terminal.

4.3.1 Program dan Struktur Dasar

Listing 4: Implementasi aturan <program> dan <program-header>

```

1 shared_ptr<ParseTreeNode> Parser::parseProgram() {
2     auto node = makeNode("<program>");
3     node->addChild(parseProgramHeader());
4     node->addChild(parseDeclarationPart());
5     node->addChild(parseCompoundStatement());
6     node->addChild(consumeTerminal(TokenType::period,
7         "Expected '.' at end of
8         program"));
9     return node;

```

```

9  }
10
11  shared_ptr<ParseTreeNode> Parser::parseProgramHeader() {
12      auto node = makeNode("<program-header>");
13      node->addChild(consumeTerminal(TokenType::programsy,
14                      "Expected 'program'"));
15      node->addChild(consumeTerminal(TokenType::ident,
16                      "Expected program name (
17                      identifier)"));
18      node->addChild(consumeTerminal(TokenType::semicolon,
19                      "Expected ';' after
20                      program name"));
21
22      return node;
23  }

```

4.3.2 Implementasi Deklarasi

Bagian deklarasi menggunakan perulangan `while` dan `do-while` untuk menangani aturan produksi dengan pengulangan (*Kleene star*). Berikut adalah implementasi untuk deklarasi konstanta dan variable:

Listing 5: Implementasi deklarasi konstanta

```

1  shared_ptr<ParseTreeNode> Parser::parseConstDeclaration() {
2      auto node = makeNode("<const-declaration>");
3      node->addChild(consumeTerminal(TokenType::constsy, "
4                      Expected 'const'"));
5
6      do {
7          node->addChild(consumeTerminal(TokenType::ident,
8                      "Expected identifier
9                      in const
10                     declaration"));
11          node->addChild(consumeTerminal(TokenType::eql,
12                      "Expected '==' in
13                      const declaration"
14                      ));
15          node->addChild(parseConstant());
16          node->addChild(consumeTerminal(TokenType::semicolon,
17                      "Expected ';' after
18                      constant"));
19      } while (check(TokenType::ident));
20
21      return node;
22  }

```

Listing 6: Implementasi deklarasi variable

```

1  shared_ptr<ParseTreeNode> Parser::parseVarDeclaration() {
2      auto node = makeNode("<var-declaration>");
3      node->addChild(consumeTerminal(TokenType::varsy, "
4                      Expected 'var'"));

```



```

4
5     do {
6         node->addChild(parseIdentifierList());
7         node->addChild(consumeTerminal(TokenType::colon,
8                               "Expected ':' in var
                                   declaration"));
9         node->addChild(parseType());
10        node->addChild(consumeTerminal(TokenType::semicolon,
11                                      "Expected ';' after
                                          var declaration"));
12    } while (check(TokenType::ident));
13
14    return node;
15 }

```

4.3.3 Implementasi Statement

Statement dalam bahasa Arion terdiri dari beberapa jenis. Implementasi fungsi `parseStatement()` menggunakan *lookahead* dan *backtracking* untuk membedakan antara `<assignment-statement>` dan `<procedure/function-call>`, karena keduanya diawali oleh token `ident`.

Listing 7: Implementasi parsing statement

```

1  shared_ptr<ParseTreeNode> Parser::parseStatement() {
2      auto node = makeNode("<statement>");
3
4      if (check(TokenType::ident)) {
5          size_t saved = pos;
6          parseVariable();
7          bool isAssign = check(TokenType::becomes);
8          pos = saved; // backtrack
9
10         if (isAssign) {
11             node->addChild(parseAssignmentStatement());
12         } else {
13             node->addChild(parseProcFuncCall());
14         }
15     } else if (check(TokenType::ifsy)) {
16         node->addChild(parseIfStatement());
17     } else if (check(TokenType::whilesy)) {
18         node->addChild(parseWhileStatement());
19     } else if (check(TokenType::forsy)) {
20         node->addChild(parseForStatement());
21     } else if (check(TokenType::repeatsy)) {
22         node->addChild(parseRepeatStatement());
23     } else if (check(TokenType::casesy)) {
24         node->addChild(parseCaseStatement());
25     } else if (check(TokenType::beginsy)) {
26         node->addChild(parseCompoundStatement());
27     }
28
29     return node;

```

30 }

4.3.4 Implementasi Ekspresi

Ekspresi dalam bahasa Arion mengikuti hierarki operator standar dengan tiga tingkatan prioritas: relasional, aditif, dan multiplikatif. Berikut adalah implementasi untuk ekspresi, simple-ekspresi, dan term:

Listing 8: Implementasi ekspresi dengan hierarki operator

```

1  shared_ptr<ParseTreeNode> Parser::parseExpression() {
2      auto node = makeNode("<expression>");
3      node->addChild(parseSimpleExpression());
4
5      if (check(TokenType::eq1) || check(TokenType::neq) ||
6          check(TokenType::gtr) || check(TokenType::geq) ||
7          check(TokenType::lss) || check(TokenType::leq)) {
8          node->addChild(makeNode(advance().toString()));
9          node->addChild(parseSimpleExpression());
10     }
11
12     return node;
13 }
14
15 shared_ptr<ParseTreeNode> Parser::parseTerm() {
16     auto node = makeNode("<term>");
17     node->addChild(parseFactor());
18
19     while (check(TokenType::times) || check(TokenType::rdiv)
20           || check(TokenType::idiv) || check(TokenType::imod)
21           || check(TokenType::andsy)) {
22         node->addChild(makeNode(advance().toString()));
23         node->addChild(parseFactor());
24     }
25
26     return node;
27 }
```

4.4 Mekanisme Disambiguasi

Terdapat beberapa kasus ambiguitas dalam grammar yang memerlukan *lookahead* dan *backtracking* untuk diselesaikan:

1. **Statement vs Assignment vs Function Call:** Ketika parser menemukan *ident*, parser menyimpan posisi token, memanggil `parseVariable()`, dan memeriksa apakah token berikutnya adalah *becomes*. Jika iya, parser melakukan *backtrack* ke posisi awal dan memanggil `parseAssignmentStatement()`. Jika tidak, parser melakukan *backtrack* dan memanggil `parseProcFuncCall()`.

2. **Factor: variable vs function call:** Dalam `parseFactor()`, setelah membaca `ident`, parser memeriksa apakah token berikutnya adalah `lparent`. Jika iya, dilakukan pemanggilan fungsi; jika tidak, diperlakukan sebagai `variable`.
3. **Range vs simple type:** Dalam `parseType()`, token `ident` bisa merupakan tipe sederhana atau awal dari `range`. Parser memeriksa apakah setelah ekspresi terdapat dua titik berurutan (`period period`) untuk menentukan interpretasi.

Listing 9: Disambiguasi factor: variable vs function call

```

1  shared_ptr<ParseTreeNode> Parser::parseFactor() {
2      auto node = makeNode("<factor>");
3
4      if (check(TokenType::intcon) || check(TokenType::realcon)
5          ||
6          check(TokenType::charcon) || check(TokenType::string)
7      ) {
8          node->addChild(makeNode(advance().toString()));
9      } else if (check(TokenType::ident)) {
10         size_t saved = pos;
11         advance();
12         if (check(TokenType::lparent)) {
13             pos = saved;
14             node->addChild(parseProcFuncCall());
15         } else {
16             pos = saved;
17             node->addChild(parseVariable());
18         }
19     } else if (match(TokenType::lparent)) {
20         node->addChild(makeNode(previous().toString()));
21         node->addChild(parseExpression());
22         node->addChild(consumeTerminal(TokenType::rparent,
23                                     "Expected ')'"));
24     } else if (match(TokenType::notsy)) {
25         node->addChild(makeNode(previous().toString()));
26         node->addChild(parseFactor());
27     } else {
28         error("Expected factor (value, identifier, or
29             expression)");
30     }
31
32     return node;
33 }
```

4.5 Penanganan Kesalahan Sintaks

Kesalahan sintaks ditangani menggunakan struktur `SyntaxError` yang merupakan turunan dari `std::runtime_error`. Struktur ini menyimpan nomor baris dan pesan kesalahan.

Listing 10: Struktur `SyntaxError` dan fungsi `error`

```

1 struct SyntaxError : public std::runtime_error {
2     int line;
3     SyntaxError(int line, const std::string &msg)
4         : std::runtime_error(msg), line(line) {}
5 };
6
7 [[noreturn]] void Parser::error(const string &msg) {
8     const Token &tok = peek();
9     ostringstream oss;
10    oss << "Syntax error on line " << tok.line << ": " << msg
11        << " (got " << tok.toString() << ")";
12    throw SyntaxError(tok.line, oss.str());
13 }

```

Pada `main.cpp`, eksepsi `SyntaxError` ditangkap dan pesan kesalahan ditampilkan ke `stderr`. Program akan mengembalikan kode keluar 1 jika terjadi kesalahan sintaks.

Listing 11: Penanganan error di `main.cpp`

```

1 try {
2     Parser parser(tokens);
3     auto tree = parser.parse();
4     tree->printToConsole();
5     tree->saveToFile(outFile.string());
6 } catch (const SyntaxError &e) {
7     std::cerr << "\n" << e.what() << "\n";
8     return 1;
9 }

```

Fungsi `expect()` dan `consumeTerminal()` menjadi garda terdepan dalam deteksi error. Ketika token yang diharapkan tidak ditemukan, parser segera menghentikan proses dan melaporkan kesalahan. Pesan error diformat dengan nomor baris dan token yang ditemukan, sehingga memudahkan pengguna dalam menelusuri sumber kesalahan.

4.6 Pencetakan dan Penyimpanan Parse Tree

Parse Tree yang dihasilkan dicetak ke terminal menggunakan format hierarkis dengan garis penghubung (*ASCII art*). Setiap level indentasi menggunakan prefix `|` untuk cabang yang memiliki saudara setelahnya dan spasi untuk cabang terakhir. Node akar dicetak tanpa prefix, sedangkan node anak menggunakan awalan `+-` untuk anak terakhir dan `|-` untuk anak lainnya.

Selain dicetak ke terminal, Parse Tree juga disimpan ke dalam berkas teks di direktori `test/output/` dengan nama yang sama dengan berkas masukan tetapi berekstensi `.txt`. Hal ini memungkinkan verifikasi hasil parsing secara otomatis maupun manual.

Listing 12: Fungsi pencetakan Parse Tree

```

1 void ParseTreeNode::print(std::ostream &os, const std::string
   &prefix,
2
3                               bool isLast, bool isRoot) const {
4     if (isRoot) {
5         os << label << "\n";
6     } else {
7
8     }
9 }

```

```

6         os << prefix;
7         os << (isLast ? "+-- " : "|-- ");
8         os << label << "\n";
9     }
10
11     string childPrefix = isRoot ? "" : prefix + (isLast ? "
        " : "| ");
12
13     for (size_t i = 0; i < children.size(); ++i) {
14         children[i]->print(os, childPrefix, i == children.
            size() - 1, false);
15     }
16 }

```

4.7 Struktur Direktori Proyek

Struktur direktori proyek untuk milestone kedua adalah sebagai berikut:

Listing 13: Tata letak direktori proyek

```

1 HBB-Tubes-IF2224-2026/
2     src/
3         main.cpp                # Titik masuk program
4         lexical/                # Lexical analyzer (milestone
5             1)
6             SourceBuffer.hpp/...
7             TokenType.hpp
8             Token.hpp/...
9             Scanner.hpp/...
10            Logger.hpp
11        syntax/                 # Syntax analyzer (milestone
12            2)
13            Parser.hpp/cpp       # Implementasi Recursive
14                Descent Parser
15            ParseTree.hpp       # Struktur data Parse Tree
16    test/
17        input/                 # Berkas masukan uji
18            pos_*.txt           # Kasus uji positif
19            neg_*.txt           # Kasus uji negatif (error)
20            edge_*.txt          # Kasus uji batas
21    doc/                       # Laporan dan dokumentasi
22    Makefile                   # Skrip kompilasi

```

5 Pengujian

Pengujian dilakukan terhadap berbagai kasus masukan untuk memverifikasi kebenaran lexer. Setiap kasus uji dijalankan dengan perintah `make run FILE=<nama_berkas>`. Keluaran pada `stdout` (daftar token) dan `stderr` (pesan kesalahan) dibandingkan dengan hasil yang diharapkan.

5.1 Kasus Uji 1: Kata Kunci dalam Berbagai Huruf Besar/Kecil

Berkas: edge_keywords_case.txt

Masukan:

```
1 BEGIN While PROGRAM
2 End If THEN
3 REPEAT downto NOT
4 DIV Mod Or
```

Keluaran yang diharapkan: beginsy, whilesy, programsy, endsy, ifsy, thensy, repeatsy, downtosy, notsy, idiv, imod, orsy — total 12 token kata kunci.



testcase1.png

Gambar 2: Hasil pengujian kasus uji 1

Analisis: Seluruh kata kunci dikenali dengan benar meskipun ditulis dengan variasi huruf besar/kecil, membuktikan bahwa mekanisme `toLowerCase()` bekerja sesuai spesifikasi.

5.2 Kasus Uji 2: Ambiguitas Bilangan Riil vs Titik

Berkas: edge_real_vs_period.txt

Masukan:

```
1 end.
2 3.14
3 42.
4 100.0
5 5 .3
```

Keluaran yang diharapkan: endsy, period, realcon (3.14), intcon (42), period, realcon (100), intcon (5), period, intcon (3) — total 9 token.



Gambar 3: Hasil pengujian kasus uji 2

Analisis: Scanner membedakan **realcon** dan **period** berdasarkan *lookahead*: titik hanya menjadi bagian bilangan riil jika diikuti digit. Kasus 42. menghasilkan **intcon** + **period** karena tidak ada digit setelah titik.

5.3 Kasus Uji 3: Charcon vs String

Berkas: edge_charcon_vs_string.txt

Masukan:

```
1 'a'
2 'ab'
3 'hello world'
4 'x'
5 ''
```

Keluaran yang diharapkan: charcon (a), string (ab), string (hello world), charcon (x), string () — total 5 token.



Gambar 4: Hasil pengujian kasus uji 3

Analisis: Literal dengan tepat satu karakter menjadi `charcon`; selainnya (termasuk literal kosong `"`) menjadi `string`. Perilaku ini sesuai dengan logika `val.length() == 1` pada `stringLiteral()`.

5.4 Kasus Uji 4: Komentar Bersarang dan Berkas Hanya Komentar

Berkas: `edge_nested_comment.txt` dan `edge_only_comments.txt`

Masukan (komentar bersarang):

```
1 { comment with { nested brace }  
2 x
```

Keluaran yang diharapkan: `ident (x)` — komentar dikonsumsi seluruhnya, hanya `x` yang tersisa.

Masukan (hanya komentar): Berkas yang hanya berisi komentar `{}` dan `(* *)`.

Keluaran yang diharapkan: Tidak ada token yang dicetak.



Gambar 5: Hasil pengujian kasus uji 4

Analisis: Komentar tidak bersarang; kurung kurawal { di dalam komentar diabaikan. Berkas yang hanya mengandung komentar menghasilkan keluaran kosong, membuktikan bahwa komentar tidak menghasilkan token.

5.5 Kasus Uji 5: Konstruksi yang Tidak Ditutup

Berkas: `edge_terminated.txt` dan `edge_terminated_comment.txt`

Masukan: String literal tanpa kutip penutup (`'hello`) dan komentar tanpa kurung penutup (`{ never ends`).

Keluaran yang diharapkan (stderr):

```
1 [line 2] Error: Unterminated string or character literal.
2 [line 2] Error: Unterminated comment.
```



Gambar 6: Hasil pengujian kasus uji 5

Analisis: Lexer melaporkan kesalahan dengan nomor baris yang akurat dan tidak mengalami *crash*. Scanner meneruskan pemrosesan setelah menemukan kesalahan (*error recovery*).

5.6 Kasus Uji 6: Operator == vs Karakter = Tunggal

Berkas: edge_eq1_vs_single_eq.txt

Masukan:

```
1 x == 5
2 y = 3
3 == ==
```

Keluaran yang diharapkan (stdout): ident (x), eql, intcon (5), ident (y), intcon (3), eql, eql.

Keluaran yang diharapkan (stderr): [line 2] Error: Unexpected character:
=



Gambar 7: Hasil pengujian kasus uji 6

Analisis: Operator `==` dikenali sebagai `eq1`, sedangkan karakter `=` tunggal (tanpa diikuti `=`) menghasilkan kesalahan karena bukan operator valid dalam Arion.

5.7 Kasus Uji 7: Operator `:=` vs `:` Diikuti Karakter Lain

Berkas: `edge_becomes_vs_colon.txt`

Masukan:

```
1 x := 10
2 y : integer
3 a :=b
4 c: d
```

Keluaran yang diharapkan: `ident (x), becomes, intcon (10), ident (y), colon, ident (integer), ident (a), becomes, ident (b), ident (c), colon, ident (d)` — total 12 token.



Gambar 8: Hasil pengujian kasus uji 7

Analisis: Mekanisme `match('=')` berhasil membedakan `:=` (becomes) dari `:` (colon). Perhatikan bahwa `integer` bukan kata kunci dalam Arion sehingga dikenali sebagai `ident`.

5.8 Ringkasan Hasil Pengujian

No.	Deskripsi Kasus Uji	Status	Kesalahan
1	Kata kunci berbagai <i>case</i>	Lulus	Tidak ada
2	Bilangan riil vs titik	Lulus	Tidak ada
3	Charcon vs string	Lulus	Tidak ada
4	Komentar bersarang & hanya komentar	Lulus	Tidak ada
5	Konstruksi tidak ditutup	Lulus	2 kesalahan (sesuai harapan)
6	<code>==</code> vs <code>=</code> tunggal	Lulus	1 kesalahan (sesuai harapan)
7	<code>:=</code> vs <code>:</code>	Lulus	Tidak ada

Tabel 1: Ringkasan hasil pengujian

Seluruh kasus uji menghasilkan keluaran yang sesuai dengan ekspektasi. Pesan kesalahan pada kasus uji 5 dan 6 muncul pada `stderr` dengan nomor baris yang tepat.

6 Akhiran

6.1 Kesimpulan

Pada milestone pertama ini, kelompok berhasil merancang dan mengimplementasikan *lexical analyzer* untuk bahasa Arion dengan hasil sebagai berikut:

1. **DFA berbasis modul** — Lexer berhasil mengimplementasikan DFA dengan pendekatan modular yang memisahkan pengenalan identifier, bilangan, string, komentar, dan operator ke dalam modul-modul independen. Seluruh 52 jenis token dapat dikenali dengan benar.
2. **Penanganan ambiguitas** — Kasus-kasus ambigu seperti perbedaan `charcon` vs `string` (berdasarkan panjang), `intcon` vs `realcon` (berdasarkan *lookahead* setelah titik), dan operator satu vs dua karakter (menggunakan `match()`) berhasil ditangani secara deterministik.
3. **Error recovery** — Lexer mampu melaporkan kesalahan leksikal (karakter tidak valid, literal/komentar tidak ditutup) tanpa menghentikan proses scanning, sehingga seluruh kesalahan dalam satu berkas dapat dilaporkan sekaligus.

6.2 Tantangan yang Dihadapi

1. **DFA vs *switch-case*** — Terdapat *tradeoff* antara implementasi DFA eksplisit (tabel transisi) dan pendekatan *switch-case* yang lebih prosedural. Pendekatan *switch-case* dipilih karena lebih mudah dibaca dan di-*debug*, meskipun kurang fleksibel untuk modifikasi pola secara otomatis.
2. **Ambiguitas `charcon` vs `string`** — Kedua tipe menggunakan pembatas yang sama (`'`), sehingga klasifikasi baru dapat dilakukan setelah seluruh isi literal terkumpul. Solusinya adalah memeriksa panjang setelah penutupan kutip.
3. **Komentar tidak menghasilkan token** — Keputusan desain untuk tidak menghasilkan token `comment` menyederhanakan antarmuka dengan parser, namun memerlukan perhatian khusus agar komentar yang tidak ditutup terdeteksi sebagai kesalahan.

Daftar Pustaka

- [1] A. V. Aho, M. S. Lam, R. Sethi, dan J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston: Pearson, 2007.
- [2] J. E. Hopcroft, R. Motwani, dan J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston: Pearson, 2006.
- [3] R. W. Sebesta, *Concepts of Programming Languages*, 12th ed. Boston: Pearson, 2019.
- [4] TutorialsPoint, "Compiler Design – Lexical Analysis," [Online]. Available: https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm. [Accessed: Apr. 2, 2026].

Appendix

Sumber Daya	Tautan
Repository GitHub	https://github.com/nicholaswisee/HBB-Tubes-IF2224-2026
Deployment	https://your-deployment-url.example.com
Diagram DFA	https://drive.google.com/file/d/1XT-esS_K76ImoWZUwJKy43QvJW_NdPOa/view

Tabel 2: Tautan repository, deployment, dan diagram

Pembagian Tugas

Nama	NIM	Kontribusi
Nathanael Imandatua Manurung	13524021	25%
Nicholas Wise Saragih Sumbayak	13524037	25%
Kurt Mikhael Purba	13524065	25%
Rava Khoman Tuah Saragih	13524107	25%

Tabel 3: Pembagian kontribusi anggota kelompok